

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Debugging & Optimization Task

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 2 – Debugging & Optimisation Task
Weightage:	35% of CIA	Total Marks:	100
CO2 Statement:	<i>Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.</i>		
Student Name:	N.SATHWIK	Reg. No.:	192525077
Section / Batch:	2/2025	Date:	22/07/2026

Code of Conduct

I N.SATHWIK(192525077) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.SATHWIK

Instructions

- This test contains 20 Debugging & Optimisation Task questions. Total: 100 Marks (20*5).
- When a student answer using Scenario-Based, in evaluation the answer should be with following five requirements: Identify the problem type and constraints, Select appropriate data structure, Design the Solution, Optimization and efficiency, Clarity of representation.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic		Focus	Q Nos	Marks
Search Data Structure, Stack		Binary Search, Stack Operations	1 - 5	25
Linked List Data Structure		List Operations	6 - 10	25
Queue Data Structures		Queue Operations	11 - 15	25
Queue Data Structures, Stack Notations		Queue Operations, Expression Evaluation	16 - 20	25
GRAND TOTAL:			100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Debugging Accuracy	20	All errors corrected; code runs flawlessly	Most errors corrected; minor issues remain	Partial correction; code runs with issues	Code fails or major errors persist
Error Identification	30	Accurately identifies all logical, syntax, and edge-case errors	Identifies most major errors	Identifies some errors but misses key issues	Struggles to identify errors
Code Quality & Readability	20	Clean, modular, well-documented, follows best practices	Mostly clean and readable	Some structure but inconsistent	Poorly written, hard to understand
Explanation & Justification	20	Clear, logical, and well-justified reasoning with complexity analysis	Good explanation with some justification	Limited explanation; lacks depth	No or incorrect explanation
Testing & Validation	10	Comprehensive test cases including edge cases	Adequate test cases	Minimal testing	No testing provided

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 2)

OPEN

✓ Completed

Review →

Debugging & Optimization Task

 10/10 answered

CO2_AT2_DEBUGGING & OPTIMIZATION TASK (Set 1)

OPEN

✓ Completed

Review →

Debugging & Optimization Task

 10/10 answered

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q10 Stack

5 marks

Point out the error?

```
PEEK(stack);
return stack[top+1]
```

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int data)
9 {
10     if (top == MAX - 1)
11     {
12         printf("Overflow\n");
13     }
14     else
15     {
16         top++;
17         stack[top] = data;
18     }
19 }
20
21 int peek()
22 {
23     if (top == -1)
24     {
25         printf("Underflow\n");
26         return -1;
27     }
28
29     return stack[top];
30 }
31
32 int main()
33 {
34     int value;
35
36     push(10);
37     push(20);
38     push(30);
39
40     value = peek();
41
42     if (value != -1)
43     {
44         printf("Top element: %d\n", value);
45     }
46
47     return 0;
48 }
```

Test Input

stdin input (optional)



Output

Top element: 30

🕒 Pending manual review

✅ Submitted

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q9 Circular Queue

5 marks

Identify the issue in the following snippet for Circular Queue in Traffic Signal.

```
rear = (rear + 1) % MAX;
queue[rear] = vehicle;
if(rear == front){
    printf("Queue Full");
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int vehicle)
10 {
11     if ((rear + 1) % MAX == front)
12     {
13         printf("Queue Full\n");
14         return;
15     }
16
17     if (front == -1)
18     {
19         front = 0;
20     }
21
22     rear = (rear + 1) % MAX;
23     queue[rear] = vehicle;
24
25     printf("Vehicle added: %d\n", vehicle);
26 }
27
28 int dequeue()
29 {
30     int data;
31
32     if (front == -1)
33     {
34         printf("Queue Empty\n");
35         return -1;
36     }
37
38     data = queue[front];
39
40     if (front == rear)
41     {
42         front = -1;
43         rear = -1;
44     }
45     else
46     {
47         front = (front + 1) % MAX;
```

Test Input

stdin input (optional)

Output

```
Vehicle added: 101
Vehicle added: 102
Vehicle added: 103
Vehicle passed: 101
```

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q8 Stack

5 marks

Point out the error.
if top == MAX:
Overflow

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int data)
9 {
10     if (top == MAX - 1)
11     {
12         printf("Overflow\n");
13     }
14     else
15     {
16         top++;
17         stack[top] = data;
18         printf("Inserted: %d\n", data);
19     }
20 }
21
22 int main()
23 {
24     push(10);
25     push(20);
26     push(30);
27
28     return 0;
29 }
```

Submitted

Test Input

stdin input (optional)

Output

Inserted: 10
Inserted: 20
Inserted: 30

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q7 Stack

5 marks

Identify the issue in the below pseudocode.

POP():

if top == 0 → Underflow

🔍 **Debugging Task:** Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int stack[MAX];
6 int top = -1;
7
8 void push(int value)
9 {
10     if (top == MAX - 1)
11     {
12         printf("Overflow\n");
13     }
14     else
15     {
16         top++;
17         stack[top] = value;
18     }
19 }
20
21 int pop()
22 {
23     if (top == -1)
24     {
25         printf("Underflow\n");
26         return -1;
27     }
28     else
29     {
30         return stack[top--];
31     }
32 }
33
34 int main()
35 {
36     int value;
37
38     push(10);
39     push(20);
40     push(30);
41
42     value = pop();
43
44     if (value != -1)
45     {
46         printf("Deleted element: %d\n", value);
47     }
48
49     return 0;
50 }
```

Test Input

stdin input (optional)

Output

Deleted element: 30

🕒 Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q6 Doubly Linked List

5 marks

Identify the error in this Doubly Linked List node deletion.

```
void deleteNode(Node* del) {
    if (del->prev != NULL)
        del->prev->next = del->next;
    if (del->next != NULL)
        del->next->prev = del->prev;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *prev;
8     struct Node *next;
9 };
10
11 struct Node *head = NULL;
12
13 void deleteNode(struct Node *del)
14 {
15     if (del == NULL)
16         return;
17
18     if (del->prev != NULL)
19         del->prev->next = del->next;
20     else
21         head = del->next;
22
23     if (del->next != NULL)
24         del->next->prev = del->prev;
25
26     free(del);
27 }
28
29 int main()
30 {
31     struct Node *a, *b, *c;
32
33     a = malloc(sizeof(struct Node));
34     b = malloc(sizeof(struct Node));
35     c = malloc(sizeof(struct Node));
36
37     a->data = 10;
38     b->data = 20;
39     c->data = 30;
40
41     a->prev = NULL;
42     a->next = b;
43
44     b->prev = a;
45     b->next = c;
```

Test Input

stdin input (optional)



Output

List after deletion: 10 30

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q5 Queue

5 marks

Identify the error in the Queue enqueue logic.

```
void enqueue(int data) {
    if (rear == MAX) return;
    queue[rear++] = data;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int deque[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void insertFront(int data)
10 {
11     if (front == -1)
12     {
13         front = 0;
14         rear = 0;
15     }
16     else if (front == 0)
17     {
18         front = MAX - 1;
19     }
20     else
21     {
22         front--;
23     }
24
25     deque[front] = data;
26 }
27
28 int main()
29 {
30     insertFront(10);
31     insertFront(20);
32
33     printf("Deque: ");
34
35     if (front != -1)
36     {
37         int i = front;
38
39         while (1)
40         {
41             printf("%d ", deque[i]);
42
43             if (i == rear)
44                 break;
45
46             i = (i + 1) % MAX;
47         }
48     }
```

Test Input

stdin input (optional)

Output

Deque: 20 10

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q4 Queue

5 marks

Find the error in this Deque insertFront operation.

```
void insertFront(int data) {
    if (front == 0) front = MAX - 1;
    else front = front - 1;
    deque[front] = data;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int deque[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void insertFront(int data)
10 {
11     if ((front == 0 && rear == MAX - 1) ||
12         (front == rear + 1))
13     {
14         printf("Overflow\n");
15         return;
16     }
17
18     if (front == -1)
19     {
20         front = 0;
21         rear = 0;
22     }
23     else if (front == 0)
24     {
25         front = MAX - 1;
26     }
27     else
28     {
29         front = front - 1;
30     }
31
32     deque[front] = data;
33 }
34
35 int main()
36 {
37     insertFront(10);
38     insertFront(20);
39
40     printf("Deque elements: ");
41
42     if (front != -1)
43     {
44         int i = front;
45
46         while (i != -1)
```

Test Input

stdin input (optional)

Output

Deque elements: 20 10

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks



Q2

Circular Queue
5 marks



Q3

Binary Search
5 marks



Q4

Queue
5 marks



Q5

Queue
5 marks



Q6

Doubly Linked List
5 marks



Q7

Stack
5 marks



Q8

Stack
5 marks



Q9

Circular Queue
5 marks



Q10

Stack
5 marks



10/10 answered

Q3 Binary Search

5 marks

Identify the bug in this Recursive Binary Search.

```
int binarySearch(int arr[], int l, int r, int x) {
    int mid = l + (r - l) / 2;
    if (arr[mid] == x) return mid;
    if (arr[mid] > x) return binarySearch(arr, l, mid, x);
    return binarySearch(arr, mid, r, x);
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int l, int r, int x)
4 {
5     if (l > r)
6         return -1;
7
8     int mid = l + (x - 1) / 2;
9
10    if (arr[mid] == x)
11        return mid;
12
13    if (arr[mid] > x)
14        return binarySearch(arr, l, mid - 1, x);
15
16    return binarySearch(arr, mid + 1, r, x);
17 }
18
19 int main()
20 {
21     int arr[] = {10, 20, 30, 40, 50};
22     int n = 5;
23     int x, result;
24
25     printf("Enter element: ");
26     scanf("%d", &x);
27
28     result = binarySearch(arr, 0, n - 1, x);
29
30     if (result == -1)
31         printf("Element not found");
32     else
33         printf("Element found at position %d", result + 1);
34
35     return 0;
36 }
```

Test Input

30

Output

Enter element: Element found at position 3

Pending manual review

Submitted

QUESTIONS

Q1
Stack Notations - Postfix
Expression
5 marks

Q2
Circular Queue
5 marks

Q3
Binary Search
5 marks

Q4
Queue
5 marks

Q5
Queue
5 marks

Q6
Doubly Linked List
5 marks

Q7
Stack
5 marks

Q8
Stack
5 marks

Q9
Circular Queue
5 marks

Q10
Stack
5 marks

10/10 answered

Q2 Circular Queue

5 marks

Identify the bug in this Circular Queue dequeue logic.

```
int dequeue() {
    if (front == -1) return -1;
    int data = queue[front];
    front = (front + 1) % MAX;
    return data;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int value)
10 {
11     if ((rear + 1) % MAX == front)
12     {
13         printf("Overflow\n");
14         return;
15     }
16
17     if (front == -1)
18         front = 0;
19
20     rear = (rear + 1) % MAX;
21     queue[rear] = value;
22 }
23
24 int dequeue()
25 {
26     int data;
27
28     if (front == -1)
29     {
30         printf("Underflow\n");
31         return -1;
32     }
33
34     data = queue[front];
35
36     if (front == rear)
37     {
38         front = -1;
39         rear = -1;
40     }
41     else
42     {
43         front = (front + 1) % MAX;
44     }
45 }
```

Test Input

stdin input (optional)

Output

Deleted: 10
Deleted: 20

Pending manual review

QUESTIONS

Q1

Stack Notations - Postfix Expression
5 marks

Q2

Circular Queue
5 marks

Q3

Binary Search
5 marks

Q4

Queue
5 marks

Q5

Queue
5 marks

Q6

Doubly Linked List
5 marks

Q7

Stack
5 marks

Q8

Stack
5 marks

Q9

Circular Queue
5 marks

Q10

Stack
5 marks

0/10 answered

Q1 Stack Notations - Postfix Expression

5 marks

Find the bug in the evaluation of a Postfix expression.

```
while (expression[i] != '\0') {
    if (isdigit(expression[i])) push(expression[i] - '0');
    else {
        int op1 = pop();
        int op2 = pop();
        // perform operation: result = op1 + op2;
        push(result);
    }
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 int stack[100];
5 int top = -1;
6
7 void push(int value)
8 {
9     stack[++top] = value;
10 }
11
12 int pop()
13 {
14     return stack[top--];
15 }
16
17 int main()
18 {
19     char expression[100];
20     int i;
21     int op1, op2, result;
22
23     printf("Enter postfix expression: ");
24     scanf("%s", expression);
25
26     for (i = 0; expression[i] != '\0'; i++)
27     {
28         if (isdigit(expression[i]))
29         {
30             push(expression[i] - '0');
31         }
32         else
33         {
34             op2 = pop();
35             op1 = pop();
36
37             if (expression[i] == '+')
38                 result = op1 + op2;
39             else if (expression[i] == '-')
40                 result = op1 - op2;
```

Test Input

stdin input (optional)

Output

Security Error: Disallowed code pattern detected.

 Pending manual review

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q9

Linked List
5 marks



Q10

Queue
5 marks



10/10 answered

Q10 Queue

5 marks

What is missing in the below pseudocode?

```
DEQUEUE(Q):
if front == -1:
    print("Underflow")
else:
    return Q[front++]
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 #define MAX 5
4
5 int queue[MAX];
6 int front = -1;
7 int rear = -1;
8
9 void enqueue(int value)
10 {
11     if (rear == MAX - 1)
12     {
13         printf("Overflow\n");
14     }
15     else
16     {
17         if (front == -1)
18             front = 0;
19
20         rear++;
21         queue[rear] = value;
22     }
23 }
24
25 int dequeue()
26 {
27     int value;
28
29     if (front == -1)
30     {
31         printf("Underflow\n");
32         return -1;
33     }
34
35     value = queue[front];
36     front++;
37
38     if (front > rear)
39     {
40         front = -1;
41         rear = -1;
42     }
43
44     return value;
45 }
46
```

Test Input

stdin input (optional)

Output

Deleted element: 10

Pending manual review

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q9

Linked List
5 marks



Q10

Queue
5 marks



Q9 Linked List

5 marks

What is the error in the below code?

```
new.next = head
```

```
head = new
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head = NULL;
13     struct Node *newNode;
14
15     newNode = (struct Node *)malloc(sizeof(struct Node));
16
17     newNode->data = 10;
18
19     newNode->next = head;
20     head = newNode;
21
22     printf("Node inserted: %d\n", head->data);
23
24     return 0;
25 }
```

Submitted

Test Input

stdin input (optional)

Output

Node inserted: 10

Pending manual review

QUESTIONS

Q1
Binary Search
5 marks



Q2
Stack
5 marks



Q3
Stack
5 marks



Q4
Stack
5 marks



Q5
Stack
5 marks



Q6
Linked List
5 marks



Q7
Stack
5 marks



Q8
Linked List Data Structures
5 marks



Q9
Linked List
5 marks



Q10
Queue
5 marks



10/10 answered

Q8 Linked List Data Structures

5 marks

Identify the errors in the following code

```
while current != NULL:
    current.next = prev
    prev = current
    current = current.next
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head, *newNode, *temp;
13     struct Node *prev = NULL;
14     struct Node *current;
15     struct Node *next;
16
17     head = NULL;
18
19     newNode = (struct Node *)malloc(sizeof(struct Node));
20     newNode->data = 10;
21     newNode->next = head;
22     head = newNode;
23
24     newNode = (struct Node *)malloc(sizeof(struct Node));
25     newNode->data = 20;
26     newNode->next = head;
27     head = newNode;
28
29     newNode = (struct Node *)malloc(sizeof(struct Node));
30     newNode->data = 30;
31     newNode->next = head;
32     head = newNode;
33
34     printf("Original List: ");
35
36     temp = head;
37
38     while (temp != NULL)
39     {
40         printf("%d ", temp->data);
41         temp = temp->next;
42     }
43
44     current = head;
45
46     while (current != NULL)
47     {
48         next = current->next;
```

Test Input

stdin input (optional)

Output

Original List: 30 20 10
Reversed List: 10 20 30

Pending manual review

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q9

Linked List
5 marks



Q10

Queue
5 marks



10/10 answered

Q7 Stack

5 marks

Point out the error.

POP(stack):

if top == 0:

print("Underflow")

else:

return stack[top--]

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top = -1;
5
6 void push(int value)
7 {
8     if (top == 4)
9     {
10         printf("Overflow\n");
11     }
12     else
13     {
14         top++;
15         stack[top] = value;
16     }
17 }
18
19 int pop()
20 {
21     if (top == -1)
22     {
23         printf("Underflow\n");
24         return -1;
25     }
26     else
27     {
28         return stack[top--];
29     }
30 }
31
32 int main()
33 {
34     int value;
35
36     push(10);
37     push(20);
38     push(30);
39
40     value = pop();
41
42     if (value != -1)
43         printf("Deleted element: %d\n", value);
44
45     return 0;
46 }
```

Test Input

stdin input (optional)

Output

Deleted element: 30

Pending manual review

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q9

Linked List
5 marks



Q10

Queue
5 marks



10/10 answered

Q6 Linked List

5 marks

What is the issue?

```
temp = head
while temp.next != NULL:
    print(temp.data)
    temp = temp.next
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *next;
8 };
9
10 int main()
11 {
12     struct Node *head, *temp;
13
14     head = (struct Node *)malloc(sizeof(struct Node));
15
16     head->data = 10;
17     head->next = (struct Node *)malloc(sizeof(struct Node));
18
19     head->next->data = 20;
20     head->next->next = NULL;
21
22     temp = head;
23
24     while (temp != NULL)
25     {
26         printf("%d ", temp->data);
27         temp = temp->next;
28     }
29
30     return 0;
31 }
```

Test Input

stdin input (optional)

Output

10 20

Pending manual review

Submitted

QUESTIONS

Q1

Binary Search
5 marks



Q2

Stack
5 marks



Q3

Stack
5 marks



Q4

Stack
5 marks



Q5

Stack
5 marks



Q6

Linked List
5 marks



Q7

Stack
5 marks



Q8

Linked List Data Structures
5 marks



Q9

Linked List
5 marks



Q10

Queue
5 marks



10/10 answered

Q5 Stack

5 marks

In balancing Bracket, you identify the error.

```
if((ch == ']' && stack[top] == '[') ||
(ch == ')' && stack[top] == '(')) {
    pop();
} else {
    return 0;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 char items[100];
4 int top = -1;
5
6 void insert(char c)
7 {
8     top++;
9     items[top] = c;
10 }
11
12 void removeItem()
13 {
14     top--;
15 }
16
17 int main()
18 {
19     char str[100];
20     int i;
21
22     printf("Enter brackets: ");
23     fgets(str, sizeof(str), stdin);
24
25     for (i = 0; str[i] != '\0'; i++)
26     {
27         if (str[i] == '(' || str[i] == '[' || str[i] == '{')
28         {
29             insert(str[i]);
30         }
31         else if (str[i] == ')' || str[i] == ']' || str[i] == '}')
32         {
33             if (top == -1)
34             {
35                 printf("Not Balanced");
36                 return 0;
37             }
38
39             if ((str[i] == ')' && items[top] == '(') ||
40                 (str[i] == ']' && items[top] == '[') ||
41                 (str[i] == '}' && items[top] == '{'))
42             {
43                 removeItem();
44             }
45         }
46     }
```

Test Input

{()}}

Output

Pending manual review

QUESTIONS

Q1
Binary Search
5 marks



Q2
Stack
5 marks



Q3
Stack
5 marks



Q4
Stack
5 marks



Q5
Stack
5 marks



Q6
Linked List
5 marks



Q7
Stack
5 marks



Q8
Linked List Data Structures
5 marks



Q9
Linked List
5 marks



Q10
Queue
5 marks



10/10 answered

Q4 Stack

5 marks

Identify the errors in the following snippet.

```
if(ch == ')') {
while(stack[top] != ')') {
postfix[j++] = pop();
}
pop();
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 char stack[100];
5 int top = -1;
6
7 void push(char ch)
8 {
9     stack[++top] = ch;
10 }
11
12 char pop()
13 {
14     return stack[top--];
15 }
16
17 int priority(char ch)
18 {
19     if (ch == '+' || ch == '-')
20         return 1;
21     if (ch == '*' || ch == '/')
22         return 2;
23     return 0;
24 }
25
26 int main()
27 {
28     char infix[100], postfix[100];
29     int i, j = 0;
30     char ch;
31
32     printf("Enter infix expression: ");
33     scanf("%s", infix);
34
35     for (i = 0; infix[i] != '\0'; i++)
36     {
37         ch = infix[i];
38
39         if (isalnum(ch))
40         {
41             postfix[j++] = ch;
42         }
43         else if (ch == '(')
44         {
45             push(ch);
46         }
47     }
```

Test Input

(a+b)*c

Output



Pending manual review

QUESTIONS

Q1
Binary Search
5 marks



Q2
Stack
5 marks



Q3
Stack
5 marks



Q4
Stack
5 marks



Q5
Stack
5 marks



Q6
Linked List
5 marks



Q7
Stack
5 marks



Q8
Linked List Data Structures
5 marks



Q9
Linked List
5 marks



Q10
Queue
5 marks



10/10 answered

Q3 Stack

5 marks

Check the snippet and identify the logical error.

```
int stack[5];
int top;
void init() {
    top = 0;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int stack[5];
4 int top;
5
6 void init()
7 {
8     top = -1;
9 }
10
11 void push(int value)
12 {
13     if (top == 4)
14     {
15         printf("Stack Overflow\n");
16     }
17     else
18     {
19         top++;
20         stack[top] = value;
21         printf("%d inserted\n", value);
22     }
23 }
24
25 void pop()
26 {
27     if (top == -1)
28     {
29         printf("Stack Underflow\n");
30     }
31     else
32     {
33         printf("%d deleted\n", stack[top]);
34         top--;
35     }
36 }
37
38 void display()
39 {
40     int i;
41
42     if (top == -1)
43     {
44         printf("Stack is empty\n");
45     }
46     else
47     {
```

Test Input

stdin input (optional)

Output

```
10 inserted
20 inserted
30 inserted
Stack: 30 20 10
30 deleted
After deletion: 20 10
```

Pending manual review

Find the error in the stack deletion operation. Justify it.

```
int pop() {  
    if(top == 0) {  
        printf("Underflow");  
        return -1;  
    }  
    return stack[top--];  
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>  
2  
3 #define MAX 5  
4  
5 int stack[MAX];  
6 int top = -1;  
7  
8 void push(int x)  
9 {  
10     if (top == MAX - 1)  
11         printf("Overflow\n");  
12     else  
13     {  
14         top++;  
15         stack[top] = x;  
16         printf("Inserted: %d\n", x);  
17     }  
18 }  
19  
20 void pop()  
21 {  
22     if (top == -1)  
23         printf("Underflow\n");  
24     else  
25     {  
26         printf("Deleted: %d\n", stack[top]);  
27         top--;  
28     }  
29 }  
30  
31 void display()  
32 {  
33     int i;  
34  
35     if (top == -1)  
36         printf("Stack is empty\n");  
37     else  
38     {  
39         for (i = top; i >= 0; i--)  
40             printf("%d ", stack[i]);  
41     }  
42 }  
43  
44 int main()
```

Test Input

```
1  
10  
1  
20
```

Output

```
Inserted: 10  
Inserted: 20  
Inserted: 30  
Stack: 30 20 10  
Deleted: 30  
After deletion: 20 10
```

 Pending manual review

QUESTIONS

Q1

Binary Search

5 marks



Q2

Stack

5 marks



Q3

Stack

5 marks



Q4

Stack

5 marks



Q5

Stack

5 marks



Q6

Linked List

5 marks



Q7

Stack

5 marks



Q8

Linked List Data Structures

5 marks



Q9

Linked List

5 marks



Q10

Queue

5 marks



10/10 answered

Q1 Binary Search

5 marks

Identify the error in the following snippet.

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while(low < high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key)
            return mid;
        else if(arr[mid] < key)
            low = mid;
        else
            high = mid;
    }
    return -1;
}
```

Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int key) {
4     int low = 0, high = n - 1;
5
6     while (low <= high) {
7         int mid = (low + high) / 2;
8
9         if (arr[mid] == key)
10             return mid;
11         else if (arr[mid] < key)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16
17     return -1;
18 }
19
20 int main() {
21     int n, key, i;
22
23     // Input
24     scanf("%d", &n);
25
26     int arr[n];
27
28
29     for (i = 0; i < n; i++) {
30         scanf("%d", &arr[i]);
31     }
32
33     scanf("%d", &key);
34
35     // Function call
36     int result = binarySearch(arr, n, key);
37 }
```

Test Input

```
5
10 20 30 40 50
30
```

Output

```
Element found at index 2
```

Pending manual review